

ADVANCED EMBEDDED SYSTEMS ASSIGNMENT

ATmega328P Power-Down Sleep Mode

Design, Register-Level Analysis, and Proteus 8 Professional Simulation

Microcontroller	ATmega328P (32-pin TQFP/QFN)
Simulation Tool	Proteus 8 Professional
Sleep Mode	Power-Down (SM2:0 = 010)
Wake Source	External Interrupt INT0 (PD2), level-triggered
Status Display	16×2 LCD (LM016L), 4-bit interface

1. Project Metadata & Executive Summary

Field	Detail
Microcontroller	ATmega328P (32-pin TQFP/QFN)
Simulation Tool	Proteus 8 Professional
Sleep Mode	Power-Down (<code>SM2:0 = 010</code>)
Wake-Up Source	External Interrupt <code>INT0</code> (PD2), level-triggered
Status Display	16×2 alphanumeric LCD (LM016L), 4-bit bit-banged interface
Clock Source	16 MHz external crystal
Deliverables	Circuit design, register-level firmware, Mermaid.js flowchart, Proteus verification screenshots

Executive Summary. This report documents a low-power embedded system built around the ATmega328P’s Power-Down sleep mode. The firmware cycles through three observable states — active operation, sleep, and interrupt-driven wake-up — using only a level-triggered external interrupt on `INT0` to resume execution, deliberately avoiding any reliance on the device’s reset line. All power-control registers (`SMCR` , `EIMSK` , `EICRA` , `EIFR` , `SREG`) are configured explicitly in source and are individually justified below. The design was verified in Proteus across all three states, with LCD state indication and a buzzer confirmation on wake-up.

2. System Objective & Low-Power Architecture

2.1 Objective

Design and simulate a practical low-power ATmega328P system that enters a genuine hardware sleep state and resumes execution **from the instruction immediately following** `sleep_cpu()` , driven exclusively by an asynchronous external interrupt — never a device reset.

2.2 Sleep Mode Comparison

The ATmega328P selects among six sleep modes via the `SM2:0` bits of `SMCR` . Each mode trades off power savings against which peripherals remain clocked and which events can wake the device.

Sleep Mode	SM2:0	Peripherals Still Active	Typical Wake Sources
Idle	<code>000</code>	All clocks except CPU/Flash; Timers, ADC, USART, SPI continue	Any enabled interrupt
ADC Noise Reduction	<code>001</code>	External oscillator, Timer2, Watchdog, ADC	ADC conversion complete, <code>INT0</code> / <code>INT1</code> , pin change, TWI, WDT
Power-Down	<code>010</code>	Only asynchronous logic: <code>INT0</code> / <code>INT1</code> , pin change, Watchdog, TWI address match	External interrupt (level), pin change, WDT, TWI
Power-Save	<code>011</code>	Timer2 (async, external 32 kHz crystal)	Timer2, external interrupt, pin change, WDT, TWI
Standby	<code>110</code>	Oscillator kept running (faster wake vs. Power-Down)	Same as Power-Down

2.3 Why Power-Down

Power-Down is the deepest standard sleep mode and was selected for this design because it produces the clearest before/after power-state contrast for demonstration purposes and wakes cleanly on a level-triggered external interrupt.

What Power-Down halts:

- CPU clock and Flash memory access
- Timer0, Timer1, and Timer2
- ADC and USART

What remains active (asynchronous-only logic):

- External Interrupt controller (`INT0` / `INT1`)
- Pin-Change Interrupt logic

- Watchdog Timer (if enabled via fuse/register)
- Brown-Out Detector (fuse-dependent)

This is the electrical reason the wake source **must be level-triggered rather than edge-triggered**: edge detection requires a running sample clock to observe a transition, which Power-Down mode does not provide. A held logic level, by contrast, can be sensed by purely asynchronous (clockless) comparator logic.

3. Hardware Schematic & Pin Mapping Analysis

3.1 Pin Assignment Summary

Function	Port/Pin	Physical Pin (32-pin package)
LED (via R2, current-limit)	PB1	13
Buzzer driver (via R3 → Q1 base)	PB0	12
Wake-up button (INT0)	PD2	32
Crystal (XTAL1/XTAL2)	PB6 / PB7	7 / 8
Reset circuit	PC6/RESET	29
LCD Register Select (RS)	PC0	23
LCD Enable (E)	PC1	24
LCD Data D4–D7	PC2–PC5	25–28

3.2 Bill of Materials (BOM)

Component	Proteus Part	Role
Microcontroller	ATMEGA328P (32-pin TQFP/QFN)	Executes sleep/wake firmware
LED	LED-RED	Visual indicator of normal operation (PB1, via R2)
Push button (wake)	BUTTON	External interrupt source on PD2/ INT0
Push button (reset)	BUTTON	Manual reset only — not the wake source
Buzzer	BUZZER	Audible wake confirmation, driven via Q1 (PB0)
Transistor	2N2222	Low-side switch driving the buzzer from PB0
16×2 LCD	LM016L	Displays RUNNING / SLEEPING / AWAKE status
Potentiometer	RV1 (10 kΩ)	LCD contrast (V _o) adjustment
Crystal	16 MHz	System clock source on PB6/PB7
Capacitors (crystal load)	C2, C3 — 22 pF	Crystal load capacitors
Capacitor (AREF)	C1 — 100 nF	AREF decoupling
Capacitor (reset debounce)	C4 — 1 nF	Reset line debounce
Resistors	R1 (10k), R2 (330Ω), R3 (1k), R4 (10k)	Pull-ups, LED current limit, transistor base drive

Design note: The reset circuit (R4, C4, and its own push button on the RESET pin) is physically and functionally isolated from the INT0 wake button. This separation is deliberate — it guarantees that a full program restart (reset) and an interrupt-based resume (wake) are never conflated during verification.

4. Register-Level Power Control Analysis

Every register the firmware touches to implement sleep and wake-up, mapped to its bit-level effect:

Register	Bit(s)	Value	Purpose / Effect
SMCR	SM2 : SM0	010	Selects Power-Down mode — halts CPU clock and all timers on sleep entry
SMCR	SE	1 → 0	Sleep Enable: set immediately

			before <code>sleep_cpu()</code> so the <code>SLEEP</code> instruction takes effect; cleared right after waking to prevent an accidental re-sleep from a stray call elsewhere in the code
<code>EIMSK</code>	<code>INT0</code>	<code>1</code>	Enables the <code>INT0</code> external interrupt line so PD2 activity can raise an interrupt request at all
<code>EICRA</code>	<code>ISC01:ISC00</code>	<code>00</code>	Configures <code>INT0</code> as low-level triggered . Required for Power-Down wake-up, since edge detection needs a running clock that Power-Down does not supply; a level is sensed asynchronously
<code>EIFR</code>	<code>INTF0</code>	cleared before enabling; set on trigger	Manually cleared prior to enabling <code>EIMSK</code> to discard any stale pending request; sets automatically when PD2 goes low; clears automatically once the ISR executes (vector table dispatch)
<code>SREG</code>	<code>I</code> (bit 7)	<code>1</code>	Global Interrupt Enable, set via <code>sei()</code> . Without this, no interrupt — including the <code>INT0</code> wake source — can fire, regardless of any other register setting
<code>MCUCR</code>	<code>PUD</code>	<code>0</code> (default, unchanged)	Not actively written in this design; left at its default so the internal pull-up enabled on PD2 via <code>PORTD</code> remains functional

Sequencing note (critical to correctness): `cli()` is issued immediately before `sleep_enable()` , and `sei()` is issued immediately after. This closes a well-known race condition between checking a wake condition and executing `SLEEP` — without it, an interrupt could theoretically arrive in the gap between enabling sleep and the `SLEEP` instruction actually executing, causing a missed wake event.

5. Program Execution Flowchart

The flowchart below replaces the original low-resolution diagram with a structured, color-coded, multi-phase Mermaid.js flowchart. Register-level transitions (`SMCR` , `SE` , ISR dispatch) are called out explicitly at each relevant node.

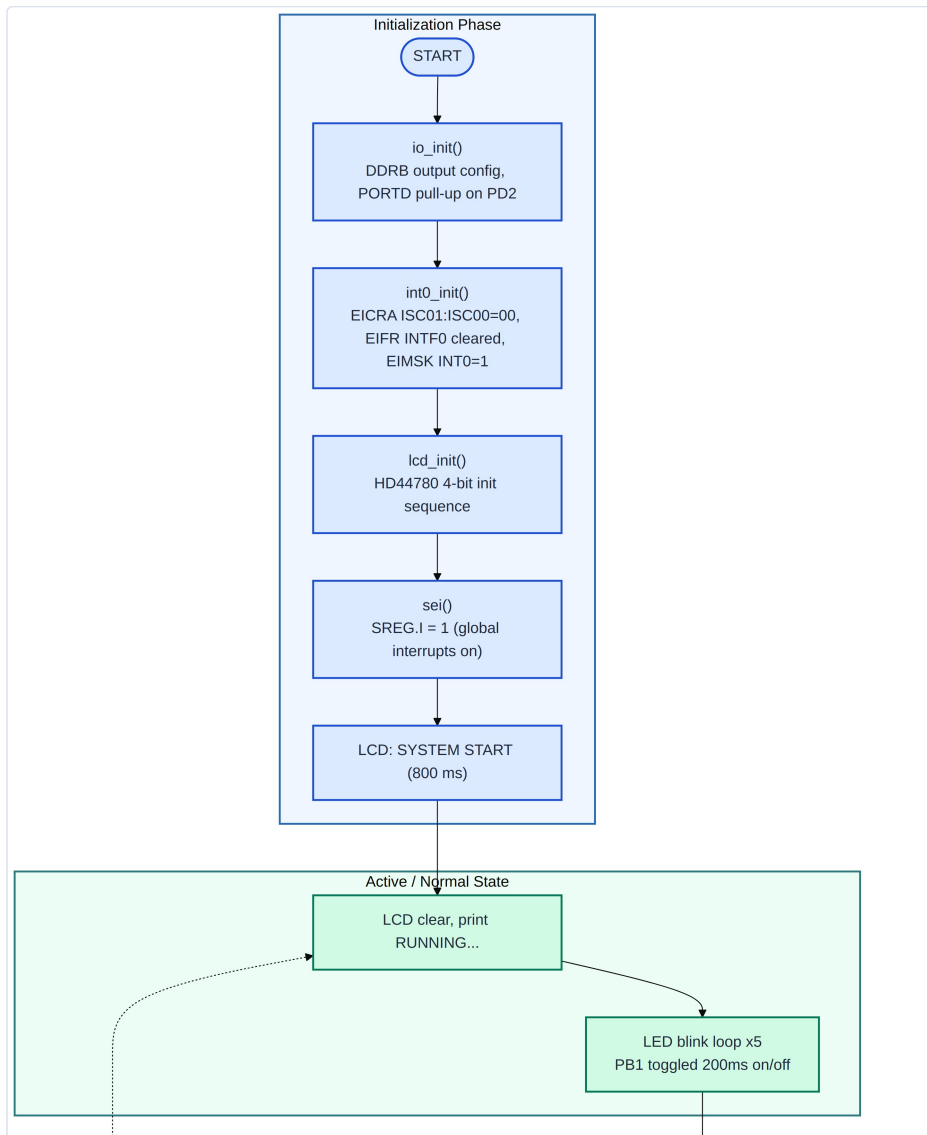


Figure 1a — Initialization Phase and Active/Normal State.

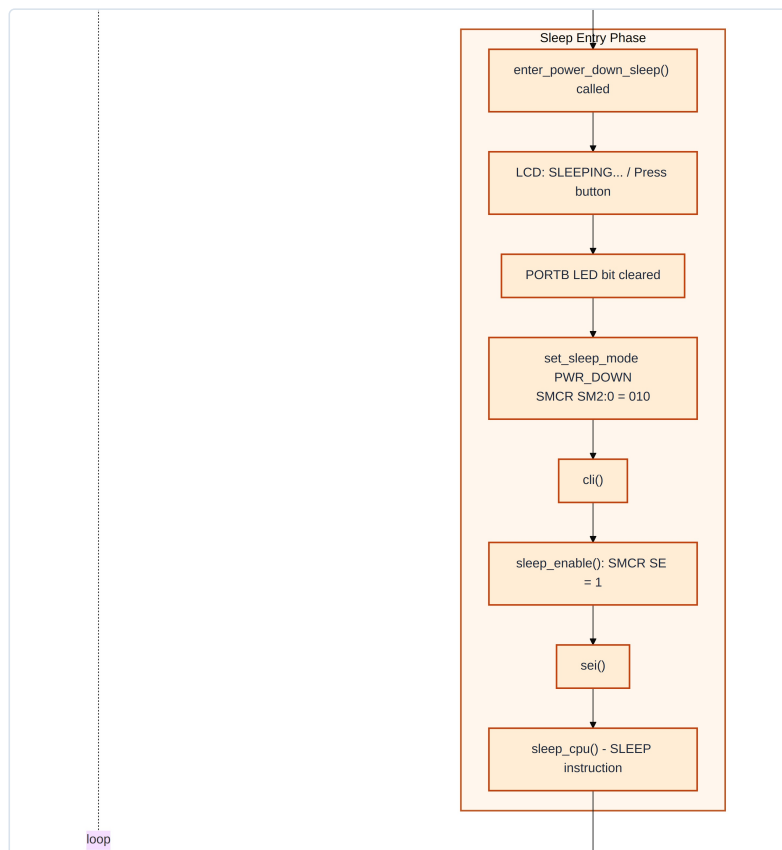


Figure 1b — Sleep Entry Phase (register writes leading up to the SLEEP instruction).

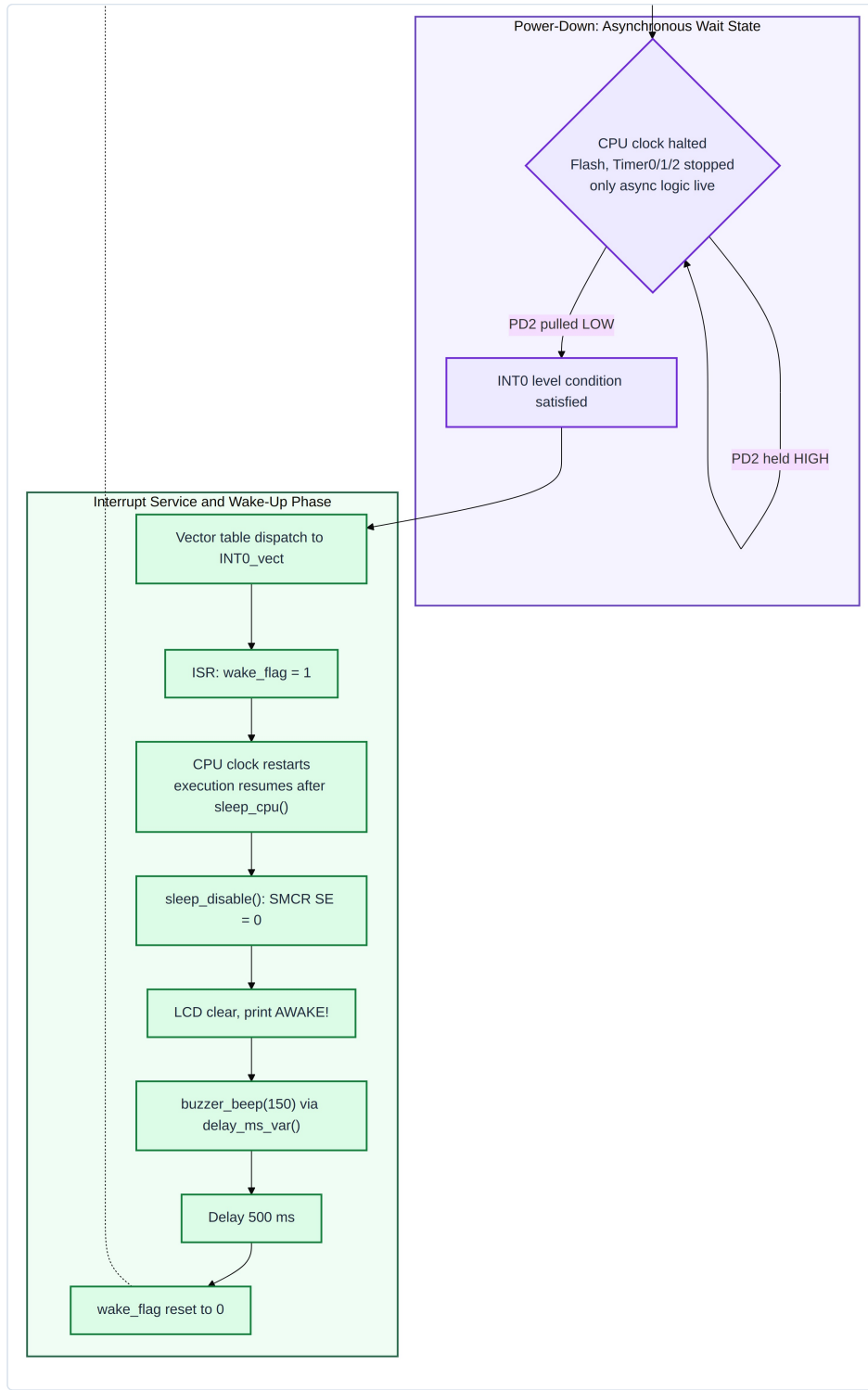


Figure 1c — Power-Down Asynchronous Wait State and Interrupt Service & Wake-Up Phase, looping back to the Active state.

Mermaid.js source (for direct reuse/editing in any Mermaid-compatible renderer):

```

graph TD
    A(["START"]) --> B["io_init()  
DDRDB output config, PORTD pull-up on PD2"]
    B --> C["int0_init()  
EICRA ISC01:ISC00=00, EIFR INTF0 cleared, EIMSK INT0=1"]
    C --> D["lcd_init()  
HD44780 4-bit init sequence"]
    D --> E["sei()  
SREG.I = 1 (global interrupts on)"]
    E --> F["LCD: SYSTEM START (800 ms)"]

    F --> G["LCD clear, print RUNNING..."]
    G --> H["LED blink loop x5  
PB1 toggled 200ms on/off"]

    H --> I["enter_power_down_sleep() called"]
    I --> J["LCD: SLEEPING... / Press button"]
    J --> K["PORTB LED bit cleared"]
    K --> L["set_sleep_mode PWR_DOWN  
SMCR SM2:0 = 010"]
    L --> M["cli()"]
    M --> N["sleep_enable(): SMCR SE = 1"]
    N --> O["sei()"]
    O --> P["sleep_cpu() - SLEEP instruction"]

    P --> Q["CPU clock halted  
Flash, Timer0/1/2 stopped  
only async logic live"]
    Q --> R["PD2 held HIGH"]
    R --> S["PD2 pulled LOW"]
    S --> T["INT0 level condition satisfied"]

    T --> U["Vector table dispatch to INT0_vect"]
    U --> V["ISR: wake_flag = 1"]
    V --> W["CPU clock restarts  
execution resumes after sleep_cpu()"]
    W --> X["sleep_disable(): SMCR SE = 0"]
    X --> Y["LCD clear, print AWAKE!"]
    Y --> Z["buzzer_beep(150) via delay_ms_var()"]
    Z --> A

    subgraph INIT [" Initialization Phase "]
        A
        B
        C
        D
        E
        F
    end

    subgraph ACTIVE [" Active / Normal State "]
        G
        H
    end

    subgraph SLEEPENTRY [" Sleep Entry Phase "]
        I
        J
        K
        L
        M
        N
        O
        P
    end

    subgraph PWRDOWN [" Power-Down: Asynchronous Wait State "]
        Q
        R
    end

    subgraph ISR_PHASE [" Interrupt Service and Wake-Up Phase "]
        S
        T
        U
        V
        W
        X
        Y
        Z
    end

```

6. Updated Embedded C Source Code & Technical Commentary

6.1 Full Source Listing

```

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/sleep.h>
#include <avr/interrupt.h>
#include <util/delay.h>

```

```

#define LED_PIN      PB1
#define BUZZER_PIN   PB0
#define BUTTON_PIN    PD2

/* ----- LCD (4-bit mode, HD44780) ----- */
#define LCD_PORT      PORTC
#define LCD_DDR        DDRC
#define LCD_RS         PC0
#define LCD_EN         PC1
#define LCD_D4         PC2
#define LCD_D5         PC3
#define LCD_D6         PC4
#define LCD_D7         PC5

static void lcd_pulse_enable(void) {
    LCD_PORT |= (1 << LCD_EN);
    _delay_us(1);
    LCD_PORT &= ~(1 << LCD_EN);
    _delay_us(100);
}

static void lcd_send_nibble(uint8_t nibble) {
    LCD_PORT &= ~((1 << LCD_D4) | (1 << LCD_D5) | (1 << LCD_D6) | (1 << LCD_D7));
    if (nibble & 0x01) LCD_PORT |= (1 << LCD_D4);
    if (nibble & 0x02) LCD_PORT |= (1 << LCD_D5);
    if (nibble & 0x04) LCD_PORT |= (1 << LCD_D6);
    if (nibble & 0x08) LCD_PORT |= (1 << LCD_D7);
    lcd_pulse_enable();
}

static void lcd_write(uint8_t value, uint8_t is_data) {
    if (is_data) LCD_PORT |= (1 << LCD_RS);
    else        LCD_PORT &= ~(1 << LCD_RS);

    lcd_send_nibble(value >> 4);
    lcd_send_nibble(value & 0x0F);
    _delay_us(50);
}

void lcd_command(uint8_t cmd) { lcd_write(cmd, 0); }
void lcd_data(uint8_t data)    { lcd_write(data, 1); }

void lcd_clear(void) {
    lcd_command(0x01);
    _delay_ms(2);
}

void lcd_set_cursor(uint8_t row, uint8_t col) {
    uint8_t addr = (row == 0) ? (0x80 + col) : (0xC0 + col);
    lcd_command(addr);
}

void lcd_print(const char *s) {
    while (*s) lcd_data((uint8_t)*s++);
}

void lcd_init(void) {
    LCD_DDR |= (1 << LCD_RS) | (1 << LCD_EN) | (1 << LCD_D4) | (1 << LCD_D5) | (1 << LCD_D6) | (1 << LCD_D7);
    _delay_ms(40);

    lcd_send_nibble(0x03); _delay_ms(5);
    lcd_send_nibble(0x03); _delay_us(150);
    lcd_send_nibble(0x03); _delay_us(150);
    lcd_send_nibble(0x02);

    lcd_command(0x28); // 4-bit, 2 lines, 5x8 font
    lcd_command(0x0C); // display on, cursor off, blink off
    lcd_command(0x06); // entry mode: increment cursor, no shift
    lcd_clear();
}

/* ----- Peripheral setup ----- */
void io_init(void) {
    DDRB |= (1 << LED_PIN) | (1 << BUZZER_PIN);
    PORTB &= ~(1 << LED_PIN) | (1 << BUZZER_PIN);

    DDRD &= ~(1 << BUTTON_PIN);
    PORTD |= (1 << BUTTON_PIN); // internal pull-up
}

void delay_ms_var(uint16_t ms) {
    while (ms >= 10) { _delay_ms(10); ms -= 10; }
    while (ms--)      { _delay_ms(1); }
}

void buzzer_beep(uint16_t ms) {
    PORTB |= (1 << BUZZER_PIN); // base driven high -> transistor on -> buzzer sounds
    delay_ms_var(ms);
}

```



```

    PORTB &= ~(1 << BUZZER_PIN);
}

/* ----- External interrupt (wake source) ----- */
void int0_init(void) {
    EICRA &= ~((1 << ISC01) | (1 << ISC00)); // low level triggers INT0
    EIFR  |= (1 << INTF0);
    EIMSK |= (1 << INT0);
}

volatile uint8_t wake_flag = 0;

ISR(INT0_vect) {
    wake_flag = 1;
}

/* ----- Sleep control ----- */
void enter_power_down_sleep(void) {
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("SLEEPING...");
    lcd_set_cursor(1, 0);
    lcd_print("Press button");
    PORTB &= ~(1 << LED_PIN);

    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    cli();
    sleep_enable();
    sei();
    sleep_cpu(); // CPU halts here until INT0 wakes it

    /* --- resumes here after ISR --- */
    sleep_disable();
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("AWAKE!");
    buzzer_beep(150);
    _delay_ms(500);
}

int main(void) {
    io_init();
    int0_init();
    lcd_init();
    sei();

    lcd_set_cursor(0, 0);
    lcd_print("SYSTEM START");
    _delay_ms(800);

    while (1) {
        lcd_clear();
        lcd_set_cursor(0, 0);
        lcd_print("RUNNING...");

        for (uint8_t i = 0; i < 5; i++) {
            PORTB |= (1 << LED_PIN);
            _delay_ms(200);
            PORTB &= ~(1 << LED_PIN);
            _delay_ms(200);
        }

        enter_power_down_sleep();
        wake_flag = 0;
    }
}

```

6.2 Block-by-Block Technical Analysis

LCD driver block (`lcd_pulse_enable` → `lcd_init`) Implements the HD44780 protocol entirely in 4-bit mode via bit-banged GPIO on `PORTC`, with no external LCD library. `lcd_send_nibble()` clears then rewrites `D4:D7` on every call rather than only toggling changed bits, which trades a few cycles for guaranteed nibble integrity. `lcd_init()` follows the datasheet-mandated “blind” initialization sequence (three 0x03 nibbles at decreasing delays, then 0x02 to commit 4-bit mode) — this sequence is required because the LCD controller cannot be assumed to be in a known state at power-on.

`io_init()` — GPIO direction and pull-up configuration Sets `DDRB` bits for `LED_PIN` (PB1) and `BUZZER_PIN` (PB0) as outputs, then clears both output-latch bits so neither the LED nor the buzzer transistor is active at boot. `DDRD` is left with `BUTTON_PIN` (PD2) as an input, and `PORTD` sets its bit to enable the **internal pull-up**, meaning the button pulls the line to ground and the resting state is logic-high — consistent with the low-level trigger configured in `EICRA`.

`delay_ms_var()` — variable-duration delay wrapper (updated in this revision) `util/delay.h`’s `_delay_ms()` requires a **compile-time constant** operand to generate accurate calibrated NOP/loop counts. Passing a runtime variable (as an earlier revision did with `_delay_ms(ms)` inside `buzzer_beep`) is not standards-correct usage of the AVR-libc delay primitive. This revision fixes that by decomposing

an arbitrary runtime `ms` value into repeated **constant** 10 ms and 1 ms calls, each of which is a valid literal argument to `_delay_ms()`. This is the single functional correction distinguishing this version from the original.

buzzer_beep() — **updated to use** `delay_ms_var()` Drives `BUZZER_PIN` high (turning on Q1 and sounding the buzzer through R3), holds it for the caller-specified duration via the now-compliant `delay_ms_var()`, then drives it low again.

into_init() — **external interrupt configuration**

- `EICRA &= ~(1<<ISC01)|(1<<ISC00)` clears both interrupt-sense-control bits for `INT0`, selecting **low-level triggered** mode (`ISC01:ISC00 = 00`) — the only sense mode capable of waking Power-Down mode, since level detection is inherently asynchronous.
- `EIFR |= (1<<INTF0)` proactively clears any stale, pre-existing interrupt flag before the interrupt is unmasked, preventing a spurious immediate ISR firing from residual state.
- `EIMSK |= (1<<INT0)` unmasks the `INT0` line at the interrupt controller level.

ISR(INT0_vect) Minimal ISR body — sets `wake_flag` and returns. Kept intentionally short, since ISR execution itself is what restarts the CPU clock coming out of Power-Down; a long ISR delays the “awake and responsive” state.

enter_power_down_sleep() — **the sleep/wake control block** 1. Updates the LCD and clears the LED **before** sleeping, so the display reflects the correct state the instant the clock halts. 2. `set_sleep_mode(SLEEP_MODE_PWR_DOWN)` writes `SM2:0 = 010` into `SMCR`. 3. `cli()` → `sleep_enable()` (sets `SMCR.SE = 1`) → `sei()`: this bracket closes the race window between arming sleep and executing the `SLEEP` opcode, guaranteeing no wake interrupt can be lost between the two. 4. `sleep_cpu()` issues the actual `SLEEP` instruction; the CPU halts here until the asynchronous `INT0` logic detects PD2 low. 5. On resume — which happens at the instruction immediately following `sleep_cpu()`, **not** at `main()` — `sleep_disable()` clears `SMCR.SE` defensively, the LCD is updated to **AWAKE!**, and the buzzer confirms the wake event audibly.

main() — **state loop** A single infinite `while(1)` loop alternates between the Active state (LCD `RUNNING...`, 5 LED blink cycles) and a call into the sleep/wake routine, with `wake_flag` reset at the top of each iteration. No reset, watchdog restart, or re-entry into `main()` occurs at any point in the wake path — the loop simply continues, which is the code-level proof that wake-up is interrupt-driven rather than reset-driven.

7. Proteus Simulation & Verification Steps

Verification requires capturing all three system states distinctly, with register and display state consistent at each. All three states below were captured directly from the Proteus 8 Professional schematic capture / animation environment.

7.1 State 1 — Normal Operation

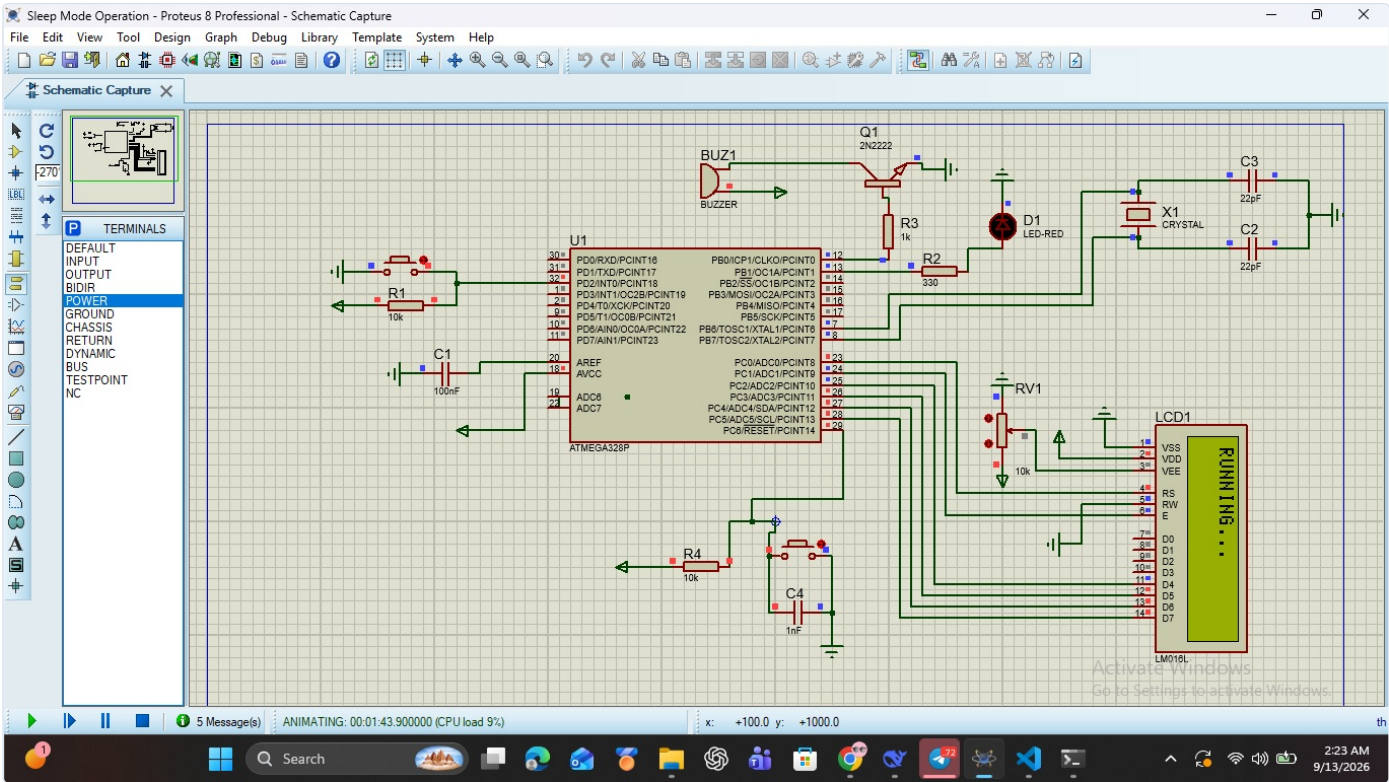


Figure 2 — Normal Operation. LCD displays 'RUNNING...' while the LED (PB1) is actively blinking; the circuit is fully clocked and executing the main loop.

Verification criteria: LCD reads `RUNNING...`; PB1 (LED) toggling visibly during animation; circuit fully clocked with the animated CPU load actively cycling (~9% in the captured run), confirming normal instruction execution.

7.2 State 2 — Sleep Mode

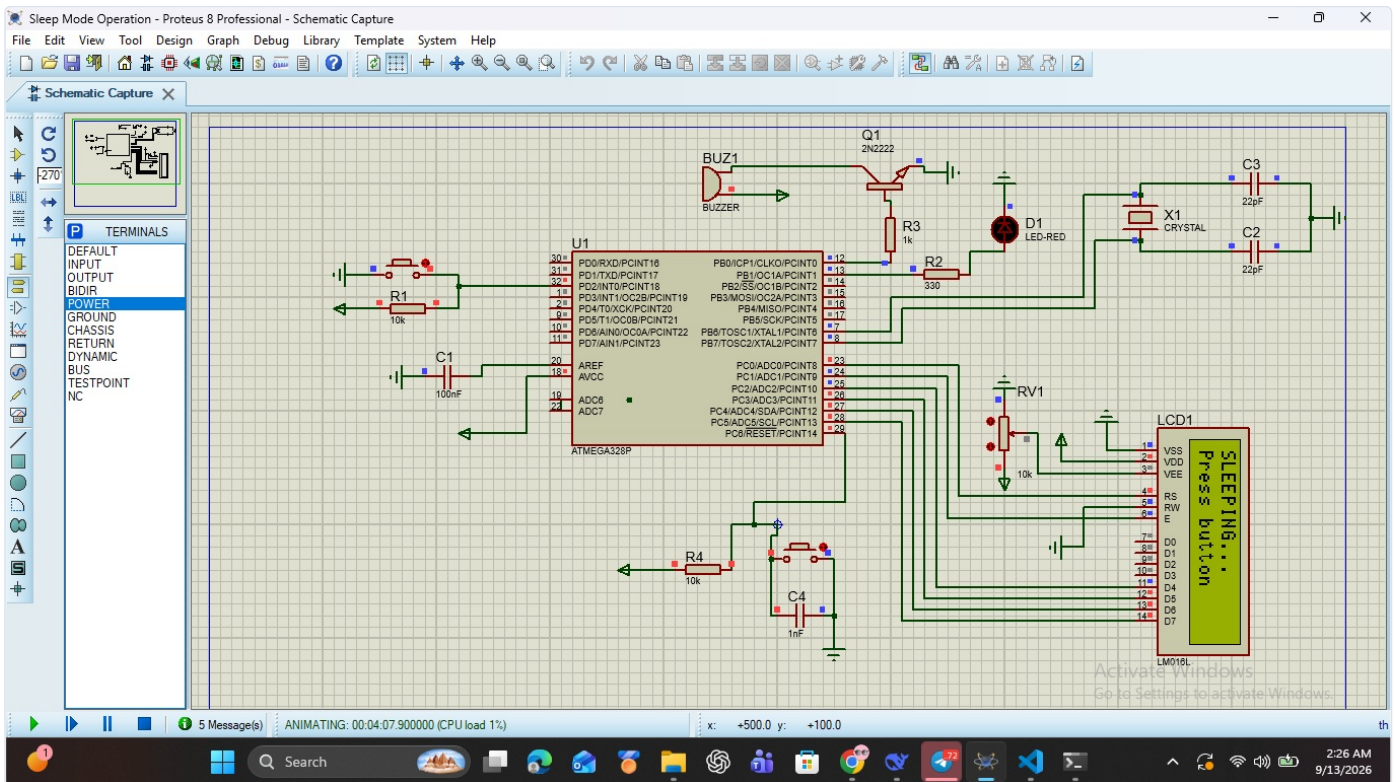


Figure 3 — Sleep Mode. LCD displays 'SLEEPING... Press button'; the LED is off. SMCR has SM2:0=010 and SE=1, and the CPU clock/timers are halted — the system is waiting only for a level change on PD2.

Verification criteria: LCD frozen on **SLEEPING...** / **Press button** ; LED (PB1) held low; **SMCR** internally holds **SM2:0 = 010** , **SE = 1** ; the simulated CPU load drops sharply (~1% in the captured run) versus the Normal Operation state, confirming the clock gating took effect.

7.3 State 3 — Interrupt-Driven Wake-Up

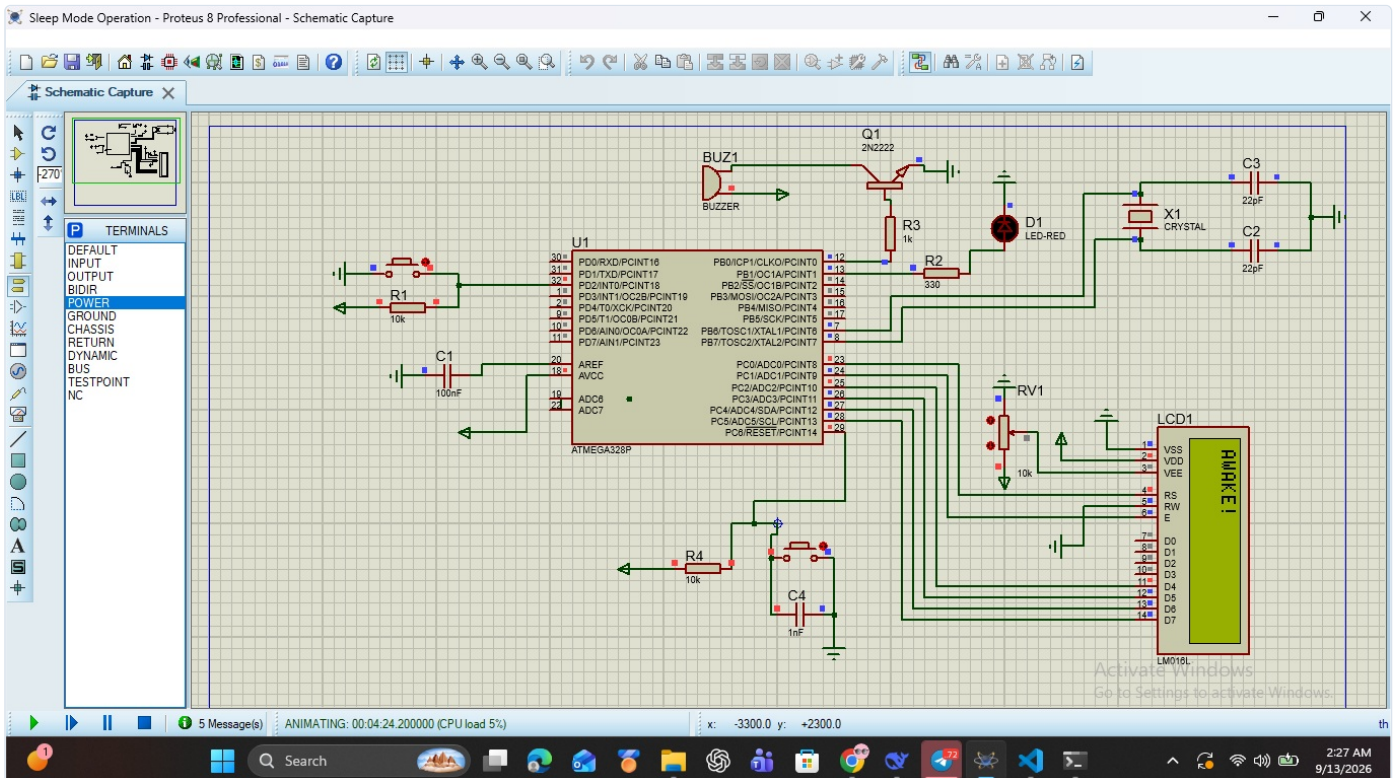


Figure 4 — Wake-up. Following a press of the button on PD2 (INT0), the LCD displays 'AWAKE!'. Execution resumed immediately after the `sleep_cpu()` call inside `enter_power_down_sleep()` — confirmed by the program continuing its normal loop afterward rather than restarting from 'SYSTEM START', which a reset would have produced.

Verification criteria: Pressing the **INT0** push button pulls PD2 low; the LCD updates to **AWAKE!** ; the buzzer pulses; program flow continues into the next loop iteration (**RUNNING...**) rather than restarting from **SYSTEM START** — this last detail is the critical evidence that a **reset did not occur**.

Key confirmation: the absence of a **SYSTEM START** reprint after wake-up is the single clearest simulation artifact distinguishing an interrupt-driven resume from a reset-driven restart, since **SYSTEM START** is only ever printed once, at the top of `main()` , prior to loop entry.

8. Engineering Reflections & Troubleshooting Notes

Crystal load capacitor selection. An early build used 22 nF load capacitors instead of the correct **22 pF** for a 16 MHz crystal — a three-orders-of-magnitude error. This produced an unstable or entirely absent oscillation, which surfaced initially as an unresponsive USART/Virtual Terminal output. Switching the status display to a directly-driven parallel LCD interface (as used throughout this design) proved more diagnostically forgiving than USART, since parallel GPIO writes are far less sensitive to clock timing/ baud accuracy than a UART peripheral is.

Asynchronous interrupt handling vs. clock availability. The most conceptually important constraint in this design is that Power-Down mode removes the system clock entirely, which eliminates edge-detection logic for external interrupts (edge detection requires successive clocked samples to identify a transition). Only level-sensing, implemented as clockless asynchronous comparator logic on `INT0 / INT1`, remains functional. This is why `EICRA` **must** be configured for low-level triggering (`ISC01:ISC00 = 00`) for any interrupt intended to wake the device from Power-Down — an edge-triggered configuration would simply never fire while the clock is halted.

Sleep/wake race condition. Bracketing `sleep_enable()` with `cli()` / `sei()` closes a narrow but real window in which an interrupt could arrive after the wake condition is armed but before `sleep_cpu()` actually executes the `SLEEP` opcode, which could otherwise cause a missed wake event or an unintended extra sleep cycle.

Variable-duration delay correctness. The original `buzzer_beep(uint16_t ms)` called `_delay_ms(ms)` with a non-constant argument, which is not valid usage of AVR-libc's calibrated busy-wait macro. The corrected implementation (`delay_ms_var()`) decomposes any runtime duration into a sequence of constant-argument `_delay_ms()` calls, preserving both timing accuracy and standards compliance.

Reset/wake isolation as a design principle. Keeping the reset push button and its RC debounce network (R4, C4) physically and functionally separate from the `INT0` wake button was a deliberate verification aid — it removes any ambiguity in Proteus screenshots about which mechanism (reset vs. interrupt) caused the MCU to resume operation.

9. Summary

This design meets the assignment's core requirement: a practical, simulated low-power ATmega328P system that enters a genuine sleep mode and wakes exclusively via an external interrupt, with all relevant registers (`SMCR` , `EIMSK` , `EICRA` , `EIFR` , `SREG`) explicitly configured, explained, and verified across three distinct, captured Proteus states.